

Homework 4

This Notebook builds on the DCOPF model introduced in [Notebook 6](#) and incorporates some elements of Economic Dispatch introduced in [Notebook 4](#).

First, load (or install if necessary) a set of packages you'll need for this assignment...

```
In [68]: #import Pkg; Pkg.add("PlotlyBase")
using JuMP
using HiGHS
using DataFrames
using CSV
using Plots; plotly();
```

Question 1: Modifying IEEE-14

A. Increased generation costs

Copy the IEEE 14 bus system and DCOPF solver function from Notebook 6. Since we neglect the resistance for the purpose of solving the DC-OPF, approximate the susceptance as:

$$B = \frac{1}{X}$$

In addition, add the following line to the return call of the function:

```
status = termination_status(DCOPF)
```

This tells you the solver termination status for the problem: e.g. was an optimal solution found, was the solution infeasible, was it unbounded, etc.

Make the following change to the system:

- Increase the variable cost of Generator 1 to \$30 / MWh

Run the DCOPF and output generation, flows, and prices.

```
In [69]: datadir = joinpath(".", "MAE 243", "ieee_test_cases")
gens = CSV.read(joinpath(datadir, "Gen14.csv"), DataFrame);
lines = CSV.read(joinpath(datadir, "Tran14.csv"), DataFrame);
loads = CSV.read(joinpath(datadir, "Load14.csv"), DataFrame);

# Rename all columns to lowercase (by convention)
for f in [gens, lines, loads]
    rename!(f, lowercase.(names(f)))
end

# create generator ids
gens.id = 1:nrow(gens);

# create line ids
lines.id = 1:nrow(lines);
```

```

# calculate simple susceptance, ignoring resistance as earlier
lines.b = 1 ./ lines.reactance

# keep only a single time period
loads = loads[:, ["connnode", "interval-1_load"]]
rename!(loads, "interval-1_load" => "demand");

```

```

In [70]: # Increase variable cost of Generator 1 to $30/MWh
gens[1, :c1] = 30.0

# Solver from Notebook 6
function dcopf_ieee(gens, lines, loads)
    DCOPF = Model(HiGHS.Optimizer)

    # Define sets based on data
    G = gens.connnode # set of generator buses

    # set of all nodes
    N = sort(union(unique(lines.fromnode),
                    unique(lines.tonode)))

    # set of all physical lines
    L = lines.id

    # Base MVA for per-unit conversion
    baseMVA = 100

    # Decision variables
    @variables(DCOPF, begin
        GEN[N]    >= 0    # generation (Pmin = 0 assumed for all resources)
        THETA[N]   # voltage phase angle at each bus
        FLOW[L]    # signed flow on each physical line
    end)

    # Slack bus: fix reference angle to 0 at bus 1
    fix(THETA[1], 0)

    # Objective function
    @objective(DCOPF, Min,
        sum(gens[g, :c1] * GEN[g] for g in G)
    )

    # Supply demand balances
    @constraint(DCOPF, cBalance[i in N],
        sum(GEN[g] for g in gens[gens.connnode .== i, :connnode])
        + sum(load for load in loads[loads.connnode .== i, :demand])
        == sum(FLOW[l] for l in lines[lines.fromnode .== i, :id]) -
        sum(FLOW[l] for l in lines[lines.tonode .== i, :id])
    )

    # Maximum generation constraints
    @constraint(DCOPF, cMaxGen[g in G],
        GEN[g] <= gens[g, :pgmax]
    )

    # Flow constraints on each branch
    @constraint(DCOPF, cLineFlows[l in L],
        FLOW[l] ==
        baseMVA * lines[l, :b] *
        (THETA[lines[l, :fromnode]] - THETA[lines[l, :tonode]])
    )

```

```

# Max line flow constraints
@constraint(DCOPF, cLineLimits[l in L],
    -lines[l, :capacity] <= FLOW[l] <= lines[l, :capacity]
)

# Solve
optimize!(DCOPF)

# Output variables
generation = DataFrame(
    node = gens.connnode,
    gen = value.(GEN).data[gens.connnode]
)

angles = value.(THETA).data

flows = DataFrame(
    id = lines.id,
    fbus = lines.fromnode,
    tbus = lines.tonode,
    flow = value.(FLOW).data
)

prices = DataFrame(
    node = N,
    value = dual.(cBalance).data
)

return (
    generation = generation,
    angles = angles,
    flows = flows,
    prices = prices,
    cost = objective_value(DCOPF),
    status = termination_status(DCOPF)
)
end

```

Out[70]: dcof_ieee (generic function with 1 method)

```

In [71]: solution = dcof_ieee(gens, lines, loads)

println("Termination status: ", solution.status)
println("\n Generation")
display(solution.generation)

println("\n Line Flows")
display(solution.flows)

println("\n Node Prices")
display(solution.prices)

```

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e+00, 2e+03]
Cost [2e+01, 3e+01]
Bound [0e+00, 0e+00]
RHS [4e+00, 1e+04]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

6 rows, 7 cols, 21 nonzeros 0s

Dependent equations search running on 6 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

6 rows, 7 cols, 21 nonzeros 0s

Presolve reductions: rows 6(-50); columns 7(-41); nonzeros 21(-103)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-7.8030701635e-03	Pr: 6(70548.3)	0.0s
6	7.0700000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal
Simplex iterations: 6
Objective value : 7.0700000000e+03
P-D objective error : 6.4316151741e-17
HiGHS run time : 0.00
Termination status: OPTIMAL

Generation

2x2 DataFrame

Row node gen

	Int64	Float64
1	1	119.0
2	2	140.0



Line Flows

20×4 DataFrame

Row	id	fbus	tbus	flow
	Int64	Int64	Int64	Float64
1	1	1	2	64.0779
2	2	1	5	54.9221
3	3	2	3	72.7846
4	4	2	4	60.9488
5	5	2	5	48.6446
6	6	3	4	-21.4154
7	7	4	5	-54.3377
8	8	4	7	29.274
9	9	4	9	16.7971
10	10	5	6	41.6289
11	11	6	11	6.03097
12	12	6	12	7.50493
13	13	6	13	16.893
14	14	7	8	-0.0
15	15	7	9	29.274
16	16	9	10	6.46903
17	17	9	14	10.102
18	18	10	11	-2.53097
19	19	12	13	1.40493
20	20	13	14	4.79796



Node Prices

14×2 DataFrame

Row	node	value
	Int64	Float64
1	1	30.0
2	2	30.0
3	3	30.0
4	4	30.0
5	5	30.0
6	6	30.0
7	7	30.0
8	8	30.0
9	9	30.0
10	10	30.0
11	11	30.0
12	12	30.0
13	13	30.0
14	14	30.0



Regarding the above results, answer the following:

- How has generation changed compared to the default system?
- What explains the new prices?

Compared to the default system generation has changed dramatically. Originally all generation occurred at node 1, but in this new formulation generation is split between node 1 (119MW) and node 2 (140MW). These changes caused the prices at each node to change, increasing them from \$20/MWh in the original model and bumping it to \$30/MWh across the system. This makes the generation change make sense as well, since the generator at node 2 only costs \$25/MWh, thus making it more economical, however since it maxes out at a production of 140MW, this forces generator 1 to cover the remaining demand. Since generator one costs more this makes the entire nodal structure pay at its price, thus \$30/MWh is found as the new nodal price across the board.

B. Constrained line

Make the following changes to the system:

- Increase the variable cost of Generator 1 to \$30 / MWh
- Reduce flow limit on the line connecting 2 and 3 (l_{23}) to 70 MW

Run the DCOPF and output generation, flows, and prices.

```
In [72]: gens_b = copy(gens)
lines_b = copy(lines)

# Increase variable cost of Generator 1 to $30/MWh
gens_b[1, :c1] = 30.0

# Reduce flow limit on Line connecting 2 and 3 ($L_{23}$) to 70 MW
lines_b[lines_b.fromnode .== 2 .&& lines_b.tonode .== 3, :capacity] .= 70.0

# Solve w/ mods
solution_b = dcopf_ieee(gens_b, lines_b, loads)

println("Termination status: ", solution_b.status)
println("\n Generation")
display(solution_b.generation)
println("\n Line Flows")
display(solution_b.flows)
println("\n Node Prices")
display(solution_b.prices)
```

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5
LP has 56 rows; 48 cols; 124 nonzeros
Coefficient ranges:
Matrix [1e+00, 2e+03]
Cost [2e+01, 3e+01]
Bound [0e+00, 0e+00]
RHS [4e+00, 1e+04]
Presolving model
27 rows, 28 cols, 85 nonzeros 0s
14 rows, 15 cols, 50 nonzeros 0s
6 rows, 7 cols, 23 nonzeros 0s
5 rows, 6 cols, 18 nonzeros 0s
3 rows, 4 cols, 9 nonzeros 0s
0 rows, 0 cols, 0 nonzeros 0s
Presolve reductions: rows 0(-56); columns 0(-48); nonzeros 0(-124) - Reduced to empty
Performed postsolve
Solving the original LP from the solution after postsolve

Model status : Optimal
Objective value : 7.5791866158e+03
P-D objective error : 5.9995534636e-17
HiGHS run time : 0.00
Termination status: OPTIMAL

Generation
2×2 DataFrame

Row	node	gen
	Int64	Float64
1	1	220.837
2	2	38.1627

Line Flows

20×4 DataFrame

Row	id	fbus	tbus	flow
	Int64	Int64	Int64	Float64
1	1	1	2	149.42
2	2	1	5	71.417
3	3	2	3	70.0
4	4	2	4	55.1212
5	5	2	5	40.7618
6	6	3	4	-24.2
7	7	4	5	-62.4868
8	8	4	7	28.9798
9	9	4	9	16.6283
10	10	5	6	42.092
11	11	6	11	6.30979
12	12	6	12	7.54588
13	13	6	13	17.0363
14	14	7	8	-0.0
15	15	7	9	28.9798
16	16	9	10	6.19021
17	17	9	14	9.91784
18	18	10	11	-2.80979
19	19	12	13	1.44588
20	20	13	14	4.98216

14×2 DataFrame

Row	node	value
	Int64	Float64
1	1	30.0
2	2	25.0
3	3	127.287
4	4	57.6793
5	5	48.8474
6	6	51.8507
7	7	56.0958
8	8	56.0958
9	9	55.2628
10	10	54.6564
11	11	53.2781
12	12	52.1203
13	13	52.331
14	14	53.9809



Regarding the above results, answer the following:

- Which node has the highest price and why?
- What is the difference in prices across l_{23} , also known as the congestion rent? How do you interpret this value (what is it's practical meaning?)

Node 3 has the highest price in this scenario. This high is also by a significant degree at \$127.287/MWh, while second place is in the upper 50s. The change of flow from node 2 to 3 to 70MW means that the load demand at node 3, which is greater than 70MW, needs to be filled via the 4 to 3 connection line. Being forced to use this line means that prices will diverge across nodes. Since this is a significant hinderance on the system where every other line maxes out at 1000 MW flow limits, this flow prevents the cheapest generator in the system from serving the downstream load, forcing the split due to the now constrained line 2 to 3. The difference in prices across l_{23} is \$102.29/MWh. This is because node 2 is at \$25/MWh and node 3 is at \$127.287/MWh, thus the difference. This value, when multiplied by the flow on the line 70MW gives us the additional rent paid by users in this scenario due to the congestion. In this case this results in \$7160.09/hr in additional costs per hour for the system.

C. Demand increase

Make the following changes to the system:

- Increase the variable cost of Generator 1 to \$30 / MWh
- Reduce flow limit on the line connecting 2 and 3 (l_{23}) to 70 MW

- Increase demands everywhere by 5%.

```
In [73]: gens_c = copy(gens)
lines_c = copy(lines)
loads_c = copy.loads)

# Increase variable cost of Generator 1 to $30/MWh
gens_c[1, :c1] = 30.0

# Reduce flow limit on the line connecting 2 and 3 ($L_{23}$) to 70 MW
lines_c[lines_c.fromnode .== 2 .&& lines_c.tonode .== 3, :capacity] . = 70.0

# Increase demands everywhere by 5%
loads_c.demand = loads_c.demand .* 1.05;
```

Calculate the total available generating capacity:

```
In [74]: total_capacity = sum(gens_c.pgmax)
println("Total available generating capacity: ", round(total_capacity, digits=2), " MW")
```

Total available generating capacity: 440.0 MW

Calculate the new total demand:

```
In [75]: total_demand = sum.loads_c.demand)
println("Total demand: ", round(total_demand, digits=2), " MW")
```

Total demand: -271.95 MW

Run the DCOPF and show prices.

```
In [76]: solution_c = dcopf_ieee(gens_c, lines_c, loads_c)

println("\nTermination status: ", solution_c.status)
println("\n Generation")
display(solution_c.generation)
println("\n Line Flows")
display(solution_c.flows)
println("\n Node Prices")
display(solution_c.prices)
```

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e+00, 2e+03]
Cost [2e+01, 3e+01]
Bound [0e+00, 0e+00]
RHS [4e+00, 1e+04]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

6 rows, 7 cols, 23 nonzeros 0s

5 rows, 6 cols, 18 nonzeros 0s

3 rows, 4 cols, 9 nonzeros 0s

Problem status detected on presolve: Infeasible

Model status : Infeasible

Objective value : 0.0000000000e+00

HiGHS run time : 0.00

Solving LP to try to compute dual ray

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e+00, 2e+03]
Cost [0e+00, 0e+00]
Bound [0e+00, 0e+00]
RHS [4e+00, 1e+04]

Solving LP without presolve or useful basis

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	0.0000000000e+00	Pr: 11(271.95); Du: 0(1.53682e-11) 0.0s
36	2.0038653229e-04	0.0s

Model status : Infeasible

Simplex iterations: 36

Objective value : 0.0000000000e+00

P-D objective error : 2.0034638557e-04

HiGHS run time : 0.00

Solving linear system to compute dual ray

Termination status: INFEASIBLE

Generation

2x2 DataFrame

Row	node	gen	
		Int64	Float64
1	1	1.5e-323	
2	2	6.95318e-310	

Line Flows

20×4 DataFrame

Row	id	fbus	tbus	flow
	Int64	Int64	Int64	Float64
1	1	1	2	1.55658e-311
2	2	1	5	6.95318e-310
3	3	2	3	1.55712e-311
4	4	2	4	1.55712e-311
5	5	2	5	1.0e-323
6	6	3	4	6.95318e-310
7	7	4	5	1.55712e-311
8	8	4	7	1.55712e-311
9	9	4	9	1.55658e-311
10	10	5	6	6.95318e-310
11	11	6	11	1.55745e-311
12	12	6	12	3.36e-322
13	13	6	13	1.55735e-311
14	14	7	8	6.95318e-310
15	15	7	9	1.55712e-311
16	16	9	10	1.55712e-311
17	17	9	14	5.0e-324
18	18	10	11	6.95318e-310
19	19	12	13	1.55745e-311
20	20	13	14	3.36e-322



Node Prices

14×2 DataFrame

Row	node		value
	Int64	Float64	
1	1	0.0	
2	2	-1.0	
3	3	19.4574	
4	4	5.53586	
5	5	3.76948	
6	6	4.37014	
7	7	5.21916	
8	8	5.21916	
9	9	5.05255	
10	10	4.93128	
11	11	4.65561	
12	12	4.42407	
13	13	4.4662	
14	14	4.79619	



What is happening in this system?

The increase to the demand has made the problem infeasible to optimize. We see this with the results where the generation, transmission, and node prices are completely unreasonable when compared to the solutions we saw in 1a and 1b as well as in consideration of being realistic. The solver is not able to satisfy the problem with the additional changes made, in particular the demand increase of 5% everywhere. The forced splitting of the system due to the I23 modification becomes too much for the added demand and the generator and transmission setup otherwise is unable to account for the demand. It is possible total system capacity is capable of meeting the demand but due to the design of the system it becomes infeasible.

Question 2: Linear losses

Up until now, we have ignored transmission losses. A quadratic approximation of losses is given by:

$$LOSS_{ij} \approx \frac{G_{ij}}{BaseMVA} (\theta_i - \theta_j)^2 \quad (1)$$

$$\approx \frac{1}{BaseMVA} \frac{R_{ij}}{R_{ij}^2 + X_{ij}^2} (\theta_i - \theta_j)^2 \quad (2)$$

where G is the line's conductance, R is the line's resistance and X is the line's reactance. See the `lines` data frame for these parameters.

For our purposes, we will approximate this quadratic via:

$$LOSS_{ij} \geq \frac{R_{ij}}{BaseMVA} \times (MaxFlow_{ij})^2 \left(\frac{|FLOW_{ij}|}{MaxFlow_{ij}} - 0.165 \right)$$

where $MaxFlow_{ij} = 200MW$ in this problem. Note the greater than equal sign, as we do not want to have negative losses.

This approximation is based on Fitiwi et al. (2016), "Finding a representative network losses model for large-scale transmission expansion planning with renewable energy sources," *Energy* 101: 343-358, <https://doi.org/10.1016/j.energy.2016.02.015>.

Note that this is a linear approximation of transmission losses, which are actually a quadratic function of power flows. Fitiwi et al. 2016 and other papers describe piece-wise or segment-wise linear approximations of the quadratic function which provide a tighter lower bound approximation of losses, but we'll use a single linear term for this assignment.

See Jenkins & Sepulveda et al. 2017, "Enhanced decision support for a changing electricity landscape: the GenX configurable electricity resource capacity expansion model", MIT Energy Initiative Working Paper 2017-10 <http://bit.ly/GenXModel> Section 5.8, for an example of a linear segment-wise approximation of quadratic transmission losses.

A. Code linear losses

Reload the original data from Notebook 6 and copy the IEEE 14 bus system and DCOPF solver function from Notebook 6 into a new function `dcopf_ieee_lossy`.

Make the following changes:

- Increase the variable cost of Generator 1 to \$30 / MWh
- Change all transmission line capacities to 200 MW

Implement losses into the supply/demand balance equations. A standard way to implement absolute values in linear programming is by introducing two non-negative auxiliary variables $x^+, x^- \geq 0$:

$$x = x^+ - x^-$$

and the absolute value can be represented as:

$$|x| = x^+ + x^-$$

(You should satisfy yourself that this equality holds.)

It makes the formulation easier if losses are added to the supply/demand balance constraint in each node by splitting losses in half between the receiving and sending end.

Indicate which equations and variables you have added and explain your steps using inline code comments (e.g. `# Comment`).

Run the lossy DCOPF and output generation, flows, losses, and prices.

```
In [77]: datadir = joinpath(".", "MAE 243", "ieee_test_cases")
gens = CSV.read(joinpath(datadir, "Gen14.csv"), DataFrame)
lines = CSV.read(joinpath(datadir, "Tran14.csv"), DataFrame)
```

```

loads = CSV.read(joinpath(datadir, "Load14.csv"), DataFrame)

# Rename all columns to Lowercase (by convention)
for f in [gens, lines, loads]
    rename!(f, lowercase.(names(f)))
end

# Create generator ids
gens.id = 1:nrow(gens)

# Create line ids
lines.id = 1:nrow(lines)

# Calculate simple susceptance, ignoring resistance:  $B = 1/X$ 
lines.b = 1 ./ lines.reactance

# Keep only a single time period
loads = loads[:, ["connnode", "interval-1_load"]]
rename!(loads, "interval-1_load" => "demand")

# Mods
gens_l = copy(gens)
lines_l = copy(lines)

# Increase variable cost of Generator 1 to $30/MWh
gens_l[1, :c1] = 30.0

# Set all transmission line capacities to 200 MW
lines_l.capacity .= 200.0

# MW – uniform line capacity used in loss approximation
MaxFlow = 200.0;

```

```

In [78]: # Lossy Solver
function dcof_ieee_lossy(gens, lines, loads)
    DCOPF = Model(HiGHS.Optimizer)

    # Define sets
    G = gens.connnode
    N = sort(union(unique(lines.fromnode), unique(lines.tonode)))
    L = lines.id

    baseMVA = 100.0

    # Decision variables
    @variables(DCOF, begin
        GEN[N] >= 0      # generation at each bus
        THETA[N]         # voltage phase angle at each bus
        FLOW[L]          # signed power flow on each line

        # NEW
        FLOW_POS[L] >= 0  # positive part of flow
        FLOW_NEG[L] >= 0  # negative part of flow
        # NEW
        LOSS[L]          >= 0  # loss on each line
    end)

    # Slack bus: fix reference angle to 0 at bus 1
    fix(THETA[1], 0)

    # Objective function

```

```

@objective(DCOPF, Min,
    sum(gens[g, :c1] * GEN[g] for g in G)
)

# Constraints
# NEW split FLOW into its positive and negative parts
@constraint(DCOPF, cFlowDecomp[l in L],
    FLOW[l] == FLOW_POS[l] - FLOW_NEG[l]
)

# NEW Linearized Loss Lower bound
@constraint(DCOPF, cLoss[l in L],
    LOSS[l] >= (lines[l, :resistance] / baseMVA) * MaxFlow^2 *
        ((FLOW_POS[l] + FLOW_NEG[l]) / MaxFlow - 0.165)
)

# MOD power balance plus losses now
@constraint(DCOPF, cBalance[i in N],
    sum(GEN[g] for g in gens[gens.connnode .== i, :connnode])
        + sum(load for load in loads[loads.connnode .== i, :demand])
    == sum(FLOW[l] for l in lines[lines.fromnode .== i, :id])
        - sum(FLOW[l] for l in lines[lines.tonode .== i, :id])
    # NEW: add half losses on lines where i is the sending end
    + sum(0.5 * LOSS[l] for l in lines[lines.fromnode .== i, :id])
    # NEW: add half losses on lines where i is the receiving end
    + sum(0.5 * LOSS[l] for l in lines[lines.tonode .== i, :id])
)

# Maximum generation constraints
@constraint(DCOPF, cMaxGen[g in G],
    GEN[g] <= gens[g, :pgmax]
)

# Flow constraints on each branch
@constraint(DCOPF, cLineFlows[l in L],
    FLOW[l] ==
        baseMVA * lines[l, :b] *
        (THETA[lines[l, :fromnode]] - THETA[lines[l, :tonode]])
)

# Max line flow constraints
@constraint(DCOPF, cLineLimits[l in L],
    -lines[l, :capacity] <= FLOW[l] <= lines[l, :capacity]
)

# Solve
optimize!(DCOPF)

# Output variables

generation = DataFrame(
    node = gens.connnode,
    gen = value.(GEN).data[gens.connnode]
)

angles = value.(THETA).data

flows = DataFrame(
    id = lines.id,
    fbus = lines.fromnode,
    tbus = lines.tonode,
    flow = value.(FLOW).data
)

```

```

)

# NEW losses for each line
losses = DataFrame(
    id      = lines.id,
    fbus    = lines.fromnode,
    tbus    = lines.tonode,
    loss    = value.(LOSS).data
)

prices = DataFrame(
    node    = N,
    value   = dual.(cBalance).data
)

return (
    generation = generation,
    flows      = flows,
    losses     = losses,
    prices     = prices,
    cost       = objective_value(DCOPF),
    status     = termination_status(DCOPF)
)
end

```

Out[78]: dcof_ieee_lossy (generic function with 1 method)

In [79]: solution_1 = dcof_ieee_lossy(gens_1, lines_1, loads)

```

println("Termination status: ", solution_1.status)
println("\n Generation")
display(solution_1.generation)
println("\n Line Flows")
display(solution_1.flows)
println("\n Losses")
display(solution_1.losses)
println("\n Node Prices")
display(solution_1.prices)
println("\nTotal generation: ", round(sum(solution_1.generation.gen), digits=2), " MW")
println("Total losses:      ", round(sum(solution_1.losses.loss),    digits=2), " MW")

```

Using BLAS: blastrampoline-5

LP has 96 rows; 108 cols; 274 nonzeros

Coefficient ranges:

Matrix [3e-02, 2e+03]
Cost [2e+01, 3e+01]
Bound [0e+00, 0e+00]
RHS [9e-01, 3e+02]

Presolving model

66 rows, 91 cols, 236 nonzeros 0s

53 rows, 78 cols, 213 nonzeros 0s

Dependent equations search running on 34 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

49 rows, 70 cols, 201 nonzeros 0s

Presolve reductions: rows 49(-47); columns 70(-38); nonzeros 201(-73)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-4.5653098671e-04	Pr: 33(6538.27); Du: 0(9.06052e-13)	0.0s
40	7.4896058230e+03	Pr: 0(0); Du: 0(1.42109e-14)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal
Simplex iterations: 40
Objective value : 7.4896058230e+03
P-D objective error : 1.2142614848e-16
HiGHS run time : 0.00
Termination status: OPTIMAL

Generation

2×2 DataFrame

Row node gen

	Int64	Float64
1	1	132.987
2	2	140.0



Line Flows

20×4 DataFrame

Row	id	fbus	tbus	flow
	Int64	Int64	Int64	Float64
1	1	1	2	72.9149
2	2	1	5	57.9503
3	3	2	3	74.2381
4	4	2	4	62.1111
5	5	2	5	49.5219
6	6	3	4	-21.8997
7	7	4	5	-55.5821
8	8	4	7	29.2291
9	9	4	9	16.7713
10	10	5	6	41.6996
11	11	6	11	6.07354
12	12	6	12	7.51118
13	13	6	13	16.9149
14	14	7	8	-0.0
15	15	7	9	29.2291
16	16	9	10	6.42646
17	17	9	14	10.0739
18	18	10	11	-2.57354
19	19	12	13	1.41118
20	20	13	14	4.82608



Losses

20×4 DataFrame

Row	id	fbus	tbus	loss
	Int64	Int64	Int64	Float64
1	1	1	2	1.5471
2	2	1	5	2.69613
3	3	2	3	3.87556
4	4	2	4	3.38329
5	5	2	5	1.88184
6	6	3	4	0.0
7	7	4	5	0.602942
8	8	4	7	0.0
9	9	4	9	0.0
10	10	5	6	0.0
11	11	6	11	0.0
12	12	6	12	0.0
13	13	6	13	0.0
14	14	7	8	0.0
15	15	7	9	0.0
16	16	9	10	0.0
17	17	9	14	0.0
18	18	10	11	0.0
19	19	12	13	0.0
20	20	13	14	0.0

14×2 DataFrame

Row	node	value
	Int64	Float64
1	1	30.0
2	2	31.0351
3	3	34.4923
4	4	34.8189
5	5	34.0158
6	6	34.2889
7	7	34.6749
8	8	34.6749
9	9	34.5992
10	10	34.544
11	11	34.4187
12	12	34.3134
13	13	34.3326
14	14	34.4826



Total generation: 272.99 MW
Total losses: 13.99 MW

B. Interpret results

Run the same parameters in the lossless OPF from problem 1. How do prices and flows change? What is the largest magnitude difference in prices between the solution with losses and the lossless OPF solution?

```
In [80]: solution_2b = dcopf_ieee(gens_l, lines_l, loads)

println("Termination status: ", solution_2b.status)
println("\n Generation")
display(solution_2b.generation)
println("\n Line Flows")
display(solution_2b.flows)
println("\n Node Prices")
display(solution_2b.prices)
println("\nTotal generation: ", round(sum(solution_2b.generation.gen), digits=2), " MW")
```

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e+00, 2e+03]
Cost [2e+01, 3e+01]
Bound [0e+00, 0e+00]
RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	-1.5856577688e-04	Pr: 7(1657.77) 0.0s
7	7.0700000000e+03	Pr: 0(0) 0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal
Simplex iterations: 7
Objective value : 7.0700000000e+03
P-D objective error : 6.4316151741e-17
HiGHS run time : 0.00
Termination status: OPTIMAL

Generation

2x2 DataFrame

Row node gen		
	Int64	Float64
1	1	119.0
2	2	140.0



Line Flows

20×4 DataFrame

Row	id	fbus	tbus	flow
	Int64	Int64	Int64	Float64
1	1	1	2	64.0779
2	2	1	5	54.9221
3	3	2	3	72.7846
4	4	2	4	60.9488
5	5	2	5	48.6446
6	6	3	4	-21.4154
7	7	4	5	-54.3377
8	8	4	7	29.274
9	9	4	9	16.7971
10	10	5	6	41.6289
11	11	6	11	6.03097
12	12	6	12	7.50493
13	13	6	13	16.893
14	14	7	8	-0.0
15	15	7	9	29.274
16	16	9	10	6.46903
17	17	9	14	10.102
18	18	10	11	-2.53097
19	19	12	13	1.40493
20	20	13	14	4.79796



Node Prices

14×2 DataFrame

Row	node	value
	Int64	Float64
1	1	30.0
2	2	30.0
3	3	30.0
4	4	30.0
5	5	30.0
6	6	30.0
7	7	30.0
8	8	30.0
9	9	30.0
10	10	30.0
11	11	30.0
12	12	30.0
13	13	30.0
14	14	30.0



Total generation: 259.0 MW

The loss model sees prices vary from the base model, node 1's prices is maintained between each model since it is directly at the source, so loss modeling is not affected. For the remaining nodes the price of transmission increases since losses incur more costs to properly meet the same amount of demand. For the flows, increases occur on lines out of bus 1, since the slack generator to make up for the losses is at bus 1. Losses consume power that is compensated by increased generation at bus 1. Otherwise, most other lines see a slight decrease in flow or nearly no change since most resistance line values are low or zero, not forcing much variation in flows. I23 despite resistance sees an large increase in flow, likely also attributed to bus 1s greater power injection, essentially overriding optimization.

The largest magnitude of price difference occurs at node 4, with the loss model showing a node price of \$34.81/MWh and the lossless model has a price of \$30/MWh.

Question 3 - Security contingencies

Power system operators need to ensure that power is delivered reliably even in the event of unexpected outages (**contingencies**). One common contingency that must be planned for is the loss of a transmission line. The security-constrained OPF (SCOPF) run by operators solves for an optimal dispatch that is simultaneously robust (i.e., feasible) to each of the lines failing individually. This is what is known as **N-1 security**, because we assume that at most one component fails in any given scenario.

In this problem, we will not code a full SCOPF, but rather investigate what happens to the feasibility of our problem when we remove transmission lines.

A. Setup data

The following code loads the original dataset (with one row per line) and includes a function `format_lines` that converts this to a format that our solver function can use (duplicating rows for both directions, adding susceptance, etc.).

```
In [81]: lines = CSV.read(joinpath(datadir, "Tran14.csv"), DataFrame)
         rename!(lines, lowercase.(names(lines)))

         function format_lines(lines)
             # create line ids
             lines.id = 1:nrow(lines)

             # calculate simple susceptance, ignoring resistance as earlier
             lines.b = 1 ./ lines.reactance
             return(lines)
         end
```

```
Out[81]: format_lines (generic function with 1 method)
```

Next:

1. Set the capacity of all lines in the system at 100 MW, except for the line l_{12} , which you should set to 200 MW.
2. Create a load dataframe `loads_sens` that increases demands everywhere by 10%

```
In [82]: lines.capacity .= 100.0
         lines[lines.fromnode .== 1 .&& lines.tonode .== 2, :capacity] .= 200.0

         loads_sens = copy(loads)
         loads_sens.demand = loads_sens.demand .* 1.10;
```

B. Loop over line contingencies

Create a dataframe `status` with the `fromnode` and `tonode` columns of `lines`.

Create a `for` loop that iterates over each line in `lines` and:

- sets the reactance to be a very high value, $1e9$ (i.e., no power will be transmitted)
- creates a version of the dataframe that our solver function can use via `format_lines`
- runs DCOPF
- stores the solution status in a `opf` column in the corresponding row of the `status` dataframe

Show the `status` results.

```
In [83]: status = DataFrame(
           fromnode = lines.fromnode,
           tonode   = lines.tonode,
           opf      = fill("", nrow(lines))
         )

         for i in 1:nrow(lines)
```

```
lines_contingency = copy(lines)

# Simulate Line outage by setting reactance to very high value
lines_contingency[i, :reactance] = 1e9

# Format Lines for solve
lines_contingency = format_lines(lines_contingency)

# Solve and mark status
try
    sol = dcopf_ieee(gens_l, lines_contingency, loads_sens)
    status[i, :opf] = string(sol.status)
catch e
    status[i, :opf] = "INFEASIBLE"
end

end

display(status)
```

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix	[1e-07, 2e+03]
Cost	[2e+01, 3e+01]
Bound	[0e+00, 0e+00]
RHS	[4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

13 rows, 14 cols, 45 nonzeros 0s

Dependent equations search running on 6 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

6 rows, 7 cols, 20 nonzeros 0s

Presolve reductions: rows 6(-50); columns 7(-41); nonzeros 20(-104)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-5.0299229368e-07	Pr: 6(424.384)	0.0s
8	7.8471810701e+03	0.0s	

Model status : Infeasible

Simplex iterations: 8

Objective value : 7.8470000000e+03

HiGHS run time : 0.00

Solving LP to try to compute dual ray

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix	[1e-07, 2e+03]
Cost	[0e+00, 0e+00]
Bound	[0e+00, 0e+00]
RHS	[4e+00, 3e+02]

Solving LP without presolve or useful basis

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	0.0000000000e+00	Pr: 11(86.9344); Du: 0(6.78495e-11)	0.0s
37	1.7645415951e-04	0.0s	

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
37	1.8143659765e-04	Pr: 2(46.4104)	0.0s
37	1.8143659765e-04	0.0s	

Model status : Unknown

Simplex iterations: 37

Objective value : 0.0000000000e+00

P-D objective error : 0.0000000000e+00

HiGHS run time : 0.00

Solving linear system to compute dual ray

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix	[1e-07, 2e+03]
Cost	[2e+01, 3e+01]
Bound	[0e+00, 0e+00]
RHS	[4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

13 rows, 14 cols, 45 nonzeros 0s

5 rows, 6 cols, 18 nonzeros 0s

4 rows, 5 cols, 13 nonzeros 0s

0 rows, 0 cols, 0 nonzeros 0s

Presolve reductions: rows 0(-56); columns 0(-48); nonzeros 0(-124) - Reduced to empty

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal
Objective value : 7.8470000000e+03
P-D objective error : 1.1589610727e-16
HiGHS run time : 0.00
Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5
LP has 56 rows; 48 cols; 124 nonzeros
Coefficient ranges:

Matrix [1e-07, 2e+03]
Cost [2e+01, 3e+01]
Bound [0e+00, 0e+00]
RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Problem status detected on presolve: Infeasible

Model status : Infeasible
Objective value : 0.0000000000e+00
HiGHS run time : 0.00

Solving LP to try to compute dual ray

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]
Cost [0e+00, 0e+00]
Bound [0e+00, 0e+00]
RHS [4e+00, 3e+02]

Solving LP without presolve or useful basis

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	0.0000000000e+00	Pr: 11(195.429); Du: 0(1.01837e-10) 0.0s
35	2.4229761597e-05	0.0s

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
35	-9.6535785227e-01	Pr: 3(1.30071e+11) 0.0s
35	-9.6535785227e-01	0.0s

Model status : Unknown

Simplex iterations: 35

Objective value : 0.0000000000e+00
P-D objective error : 0.0000000000e+00
HiGHS run time : 0.04

No dual ray found

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]
Cost [2e+01, 3e+01]
Bound [0e+00, 0e+00]
RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Problem status detected on presolve: Infeasible

Model status : Infeasible
Objective value : 0.0000000000e+00
HiGHS run time : 0.00

Solving LP to try to compute dual ray

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [0e+00, 0e+00]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Solving LP without presolve or useful basis

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	0.0000000000e+00	Pr: 11(103.359); Du: 0(1.51603e-10) 0.0s
38	1.5155315311e-04	0.0s

Model status : Infeasible

Simplex iterations: 38

Objective value : 0.0000000000e+00

P-D objective error : 1.5153018823e-04

HiGHS run time : 0.00

Solving linear system to compute dual ray

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

13 rows, 14 cols, 47 nonzeros 0s

5 rows, 6 cols, 20 nonzeros 0s

Dependent equations search running on 5 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

5 rows, 6 cols, 20 nonzeros 0s

Presolve reductions: rows 5(-51); columns 6(-42); nonzeros 20(-104)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	-1.8847596447e-07	Pr: 5(101.899) 0.0s
5	7.8470000000e+03	Pr: 0(0) 0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 5

Objective value : 7.8470000000e+03

P-D objective error : 1.1589610727e-16

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

Problem status detected on presolve: Infeasible

Model status : Infeasible

Objective value : 0.0000000000e+00

HiGHS run time : 0.00

Solving LP to try to compute dual ray

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [0e+00, 0e+00]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Solving LP without presolve or useful basis

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	0.0000000000e+00	Pr: 11(181.363); Du: 0(8.02723e-11)	0.0s
36	5.9533364321e-05		0.0s

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
36	1.2524022443e-04	Pr: 1(3.62)	0.0s
36	1.2524022443e-04		0.0s

Model status : Unknown

Simplex iterations: 36

Objective value : 0.0000000000e+00

P-D objective error : 0.0000000000e+00

HiGHS run time : 0.01

No dual ray found

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

6 rows, 7 cols, 21 nonzeros 0s

Dependent equations search running on 6 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

6 rows, 7 cols, 21 nonzeros 0s

Presolve reductions: rows 6(-50); columns 7(-41); nonzeros 21(-103)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-6.4862099867e-03	Pr: 6(103143)	0.0s
8	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 8

Objective value : 7.8470000000e+03

P-D objective error : 1.1589610727e-16

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s
Dependent equations search running on 7 equations with time limit of 1000.00s
Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)
7 rows, 8 cols, 25 nonzeros 0s
Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)
Solving the presolved LP
Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	-8.6846694107e-05	Pr: 7(935.988) 0.0s
8	7.8470000000e+03	Pr: 0(0) 0.0s

Performed postsolve
Solving the original LP from the solution after postsolve

Model status : Optimal
Simplex iterations: 8
Objective value : 7.8470000000e+03
P-D objective error : 0.0000000000e+00
HiGHS run time : 0.00
Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5
LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix	[1e-07, 2e+03]
Cost	[2e+01, 3e+01]
Bound	[0e+00, 0e+00]
RHS	[4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s
15 rows, 16 cols, 56 nonzeros 0s
Dependent equations search running on 11 equations with time limit of 1000.00s
Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)
11 rows, 12 cols, 42 nonzeros 0s
Presolve reductions: rows 11(-45); columns 12(-36); nonzeros 42(-82)
Solving the presolved LP
Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	-1.7474019111e-05	Pr: 11(51213.9); Du: 0(3.38723e-12) 0.0s
12	7.8470000000e+03	Pr: 0(0) 0.0s

Performed postsolve
Solving the original LP from the solution after postsolve

Model status : Optimal
Simplex iterations: 12
Objective value : 7.8470000000e+03
P-D objective error : 5.7948053633e-17
HiGHS run time : 0.00
Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5
LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix	[1e-07, 2e+03]
Cost	[2e+01, 3e+01]
Bound	[0e+00, 0e+00]
RHS	[4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s
14 rows, 15 cols, 50 nonzeros 0s
Dependent equations search running on 7 equations with time limit of 1000.00s
Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)
7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-8.6846694107e-05	Pr: 7(1048)	0.0s
7	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 7

Objective value : 7.8470000000e+03

P-D objective error : 0.0000000000e+00

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 52 nonzeros 0s

9 rows, 10 cols, 32 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-8.6846694107e-05	Pr: 7(976.007)	0.0s
8	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 8

Objective value : 7.8470000000e+03

P-D objective error : 1.1589610727e-16

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-8.6846694107e-05	Pr: 7(975.588)	0.0s
8	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 8

Objective value : 7.8470000000e+03

P-D objective error : 5.7948053633e-17

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-8.6846694107e-05	Pr: 7(980.2)	0.0s
8	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 8

Objective value : 7.8470000000e+03

P-D objective error : 5.7948053633e-17

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-8.6846694107e-05	Pr: 7(974.577)	0.0s
8	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 8

Objective value : 7.8470000000e+03

P-D objective error : 1.1589610727e-16

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
-----------	-----------	--------------------------

0	-8.6846694107e-05	Pr: 7(935.988) 0.0s
---	-------------------	---------------------

8	7.8470000000e+03	Pr: 0(0) 0.0s
---	------------------	---------------

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 8

Objective value : 7.8470000000e+03

P-D objective error : 5.7948053633e-17

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix [1e-07, 2e+03]

Cost [2e+01, 3e+01]

Bound [0e+00, 0e+00]

RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
-----------	-----------	--------------------------

0	-8.6846694107e-05	Pr: 7(959.156) 0.0s
---	-------------------	---------------------

8	7.8470000000e+03	Pr: 0(0) 0.0s
---	------------------	---------------

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

```

Simplex iterations: 8
Objective value      : 7.8470000000e+03
P-D objective error  : 5.7948053633e-17
HiGHS run time      : 0.00
Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5
LP has 56 rows; 48 cols; 124 nonzeros
Coefficient ranges:
  Matrix [1e-07, 2e+03]
  Cost   [2e+01, 3e+01]
  Bound  [0e+00, 0e+00]
  RHS    [4e+00, 3e+02]
Presolving model
27 rows, 28 cols, 85 nonzeros 0s
14 rows, 15 cols, 50 nonzeros 0s
Dependent equations search running on 7 equations with time limit of 1000.00s
Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)
7 rows, 8 cols, 25 nonzeros 0s
Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)
Solving the presolved LP
Using dual simplex solver
  Iteration      Objective      Infeasibilities num(sum)
          0      -8.6846694107e-05 Pr: 7(959.526) 0.0s
          8       7.8470000000e+03 Pr: 0(0) 0.0s

```

Performed postsolve
Solving the original LP from the solution after postsolve

```

Model status      : Optimal
Simplex iterations: 8
Objective value    : 7.8470000000e+03
P-D objective error: 0.0000000000e+00
HiGHS run time    : 0.00
Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5
LP has 56 rows; 48 cols; 124 nonzeros
Coefficient ranges:
  Matrix [1e-07, 2e+03]
  Cost   [2e+01, 3e+01]
  Bound  [0e+00, 0e+00]
  RHS    [4e+00, 3e+02]
Presolving model
27 rows, 28 cols, 85 nonzeros 0s
14 rows, 15 cols, 50 nonzeros 0s
Dependent equations search running on 7 equations with time limit of 1000.00s
Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)
7 rows, 8 cols, 25 nonzeros 0s
Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)
Solving the presolved LP
Using dual simplex solver
  Iteration      Objective      Infeasibilities num(sum)
          0      -8.6846694107e-05 Pr: 7(971.289) 0.0s
          8       7.8470000000e+03 Pr: 0(0) 0.0s

```

Performed postsolve
Solving the original LP from the solution after postsolve

```

Model status      : Optimal
Simplex iterations: 8
Objective value    : 7.8470000000e+03
P-D objective error: 1.1589610727e-16
HiGHS run time    : 0.00

```

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5
LP has 56 rows; 48 cols; 124 nonzeros
Coefficient ranges:
Matrix [1e-07, 2e+03]
Cost [2e+01, 3e+01]
Bound [0e+00, 0e+00]
RHS [4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-8.6846694107e-05	Pr: 7(974.677)	0.0s
8	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 8

Objective value : 7.8470000000e+03

P-D objective error : 5.7948053633e-17

HiGHS run time : 0.00

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix	[1e-07, 2e+03]
Cost	[2e+01, 3e+01]
Bound	[0e+00, 0e+00]
RHS	[4e+00, 3e+02]

Presolving model

27 rows, 28 cols, 85 nonzeros 0s

14 rows, 15 cols, 50 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

7 rows, 8 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-49); columns 8(-40); nonzeros 25(-99)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-8.6846694107e-05	Pr: 7(976.28)	0.0s
8	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 8

Objective value : 7.8470000000e+03

P-D objective error : 5.7948053633e-17

HiGHS run time : 0.00

20×3 DataFrame

Row	fromnode	tonode	opf
	Int64	Int64	String
1	1	2	INFEASIBLE
2	1	5	OPTIMAL
3	2	3	INFEASIBLE
4	2	4	INFEASIBLE
5	2	5	OPTIMAL
6	3	4	INFEASIBLE
7	4	5	OPTIMAL
8	4	7	OPTIMAL
9	4	9	OPTIMAL
10	5	6	OPTIMAL
11	6	11	OPTIMAL
12	6	12	OPTIMAL
13	6	13	OPTIMAL
14	7	8	OPTIMAL
15	7	9	OPTIMAL
16	9	10	OPTIMAL
17	9	14	OPTIMAL
18	10	11	OPTIMAL
19	12	13	OPTIMAL
20	13	14	OPTIMAL



3. Interpret results

Are all of the cases feasible? If not, how many are infeasible?

The cases are not all feasible, these results show that there are 4 infeasible results, those from nodes 1 to 2, 2 to 3, 2 to 4, and 3 to 4.

Pick two cases where the solution gives a different status. (For our purposes, dual infeasible and primal infeasible are the same.) What is happening here?

Given this, do you conclude that the system with the assumed transmission line ratings is secure as-is, or do we need to add more redundancy to the system?

When the solution gives a different status than optimal, like I12, we see that a failure in this line would take out most of the power coming out of bus 1, I15 would be forced to carry the transmission responsibilities of

l12, which is not possible if the entire demand is to be served. Additionally, in l23, we see that infeasibility also restricts flow to buses downstream and will lead to major network security issues for most of the remaining buses in the system. Based on all of this, I would conclude that the system is not secure as is. A secure system would need to maintain optimal solutions and operability across the board even in extreme scenarios like this. These scenarios showcase that the ceiling for stability of the system is quite low, as a 10% increase in load and single line failures stresses the entire system heavily and possibly shuts down operations. More redundancy, particularly in the regions we saw failures (coming out of bus 1 and the l23 l24 connections) to ensure those points don't become bottlenecks are very needed if security is to be guaranteed.

Question 4: LLM Exercise

We will now check if adding a new line in the system will reduce the security contingencies in an existing line. Create a new section at the end of your Homework notebook titled "Question 4: LLM Exercise". Include the LLM prompts used (as markdown text cells in your notebook) and ensure all code is replicated in the notebook (i.e., do not have the LLM run the code or perform any computation outside of the notebook).

a) Start from the modified setup of capacities from Question 3. Now, create a new lines dataframe with a new transmission line of capacity 100 MW between nodes 1 and 12. Copy the line parameters (reactance, resistance, susceptance, etc.) from the line from node 1 to node 2. Then remove the line from node 2 to node 4.

b) Evaluate the resulting OPF. Is it feasible? Why or why not?

I utilized Claude Sonnet 4.6 for this portion of the assignment, the free version I uploaded the HTML version of my code thus far to provide context for the assignment, my prompt was an attachment of the file with prompt:

Utilize the attached file as a guideline for code generation for a project I am working on. Using the file and my prompts I want you to generate code that achieves my goals

I then uploaded the question 4 prompt and instructions about formatting of the prompt and results, the prompt was as follows:

We will now check if adding a new line in the system will reduce the security contingencies in an existing line. a) Start from the modified setup of capacities from Question 3. Now, create a new lines dataframe with a new transmission line of capacity 100 MW between nodes 1 and 12. Copy the line parameters (reactance, resistance, susceptance, etc.) from the line from node 1 to node 2. Then remove the line from node 2 to node 4. b) Evaluate the resulting OPF. Is it feasible? Why or why not?

This produced the code you see below (2 plotting blocks and 1 solving block),

```
In [84]: # — Part 4A: Create new lines dataframe —————

lines_new = copy(lines)

# Get parameters from line l_12 to copy to new line
l12 = lines_new[lines_new.fromnode == 1 && lines_new.tonode == 2, :]

# Create new line from node 1 to node 12 with same parameters as l_12
new_line = copy(l12)
```

```

new_line.fromnode .= 1
new_line.tonode   .= 12
new_line.capacity .= 100.0

# Append new Line and remove L_24
append!(lines_new, new_line)
lines_new = lines_new[.!((lines_new.fromnode .== 2 .&& lines_new.tonode .== 4), :)]

# Reformat lines (reindex IDs and recompute susceptance)
lines_new = format_lines(lines_new)

# — Part 4B: Evaluate OPF —————

sol_new = dcopf_ieee(gens_l, lines_new, loads_sens)

println("Termination status: ", sol_new.status)
println("\n— Generation —")
display(sol_new.generation)
println("\n— Line Flows —")
display(sol_new.flows)
println("\n— Nodal Prices (LMPs) —")
display(sol_new.prices)

```

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 56 rows; 48 cols; 124 nonzeros

Coefficient ranges:

Matrix	[1e+00, 2e+03]
Cost	[2e+01, 3e+01]
Bound	[0e+00, 0e+00]
RHS	[4e+00, 3e+02]

Presolving model

28 rows, 29 cols, 86 nonzeros 0s

15 rows, 16 cols, 61 nonzeros 0s

Dependent equations search running on 13 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

13 rows, 14 cols, 55 nonzeros 0s

Presolve reductions: rows 13(-43); columns 14(-34); nonzeros 55(-69)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-2.7645751293e-03	Pr: 13(3250.73)	0.0s
13	7.8470000000e+03	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 13

Objective value : 7.8470000000e+03

P-D objective error : 5.7948053633e-17

HiGHS run time : 0.00

Termination status: OPTIMAL

— Generation —

2×2 DataFrame

Row	node	gen
	Int64	Float64
1	1	144.9
2	2	140.0



— Line Flows —

20×4 DataFrame

Row	id	fbus	tbus	flow
	Int64	Int64	Int64	Float64
1	1	1	2	23.7809
2	2	1	5	49.3353
3	3	2	3	84.7198
4	4	2	5	55.1911
5	5	3	4	-18.9002
6	6	4	5	-93.6325
7	7	4	7	14.0758
8	8	4	9	8.07654
9	9	5	6	2.53387
10	10	6	11	20.3959
11	11	6	12	-28.9077
12	12	6	13	-1.27432
13	13	7	8	-0.0
14	14	7	9	14.0758
15	15	9	10	-6.64589
16	16	9	14	-3.65176
17	17	10	11	-16.5459
18	18	12	13	36.1661
19	19	13	14	20.0418
20	20	1	12	71.7838



— Nodal Prices (LMPs) —

14×2 DataFrame

Row	node	value
	Int64	Float64
1	1	30.0
2	2	30.0
3	3	30.0
4	4	30.0
5	5	30.0
6	6	30.0
7	7	30.0
8	8	30.0
9	9	30.0
10	10	30.0
11	11	30.0
12	12	30.0
13	13	30.0
14	14	30.0



The resulting OPF is feasible. The inclusion of the new line l112 allows for a secondary route for power to flow into the other half of the network that was constrained to l12 originally. The removal of the l24 line preemptively accepts the lines failure by removing it from the problem. These changes resolve issues that were identified in part 3 as problem points and help relieve congestion and stress on valuable lines.